Real-Time Computer Systems and Computer Architecture

Laboratory 1

Registers, GPIO, Timers and Interrupts

Student: Student Name

Glasgow ID: Glasgow ID

UESTC ID: UESTC ID

September 4, 2026

1 Task 1: GPIOB Pin 5 Register Configuration

The task uses the four-bit `GPIOx_CRL` pin field described in the laboratory brief [1] and the STM32F1 reference manual [4]. Pin 5 occupies bits 23:20. General-purpose push-pull output mode requires `CNF[1:0] = 00`, and the 50 MHz output selection requires `MODE[1:0] = 11`; therefore the complete nibble is `0011`.

1.1 Step-by-step bitwise operations

Let the untouched register bits be written as x. The field mask is

$$M = 0xF \ll 20 = \texttt{0x00F00000}.$$

The starting register is represented symbolically as

xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx.

First, clear bits 23:20:

```
GPIOB->CRL &= ~0x00F00000U;
```

The result is

xxxxxxxx 0000xxxx xxxxxxxx xxxxxxxx.

Second, place `0011` in the field:

```
GPIOB->CRL |= 0x00300000U;
```

The result is

xxxxxxxx 0011xxxx xxxxxxxx xxxxxxxx.

Figure 1 shows that only the PB5 field changes.

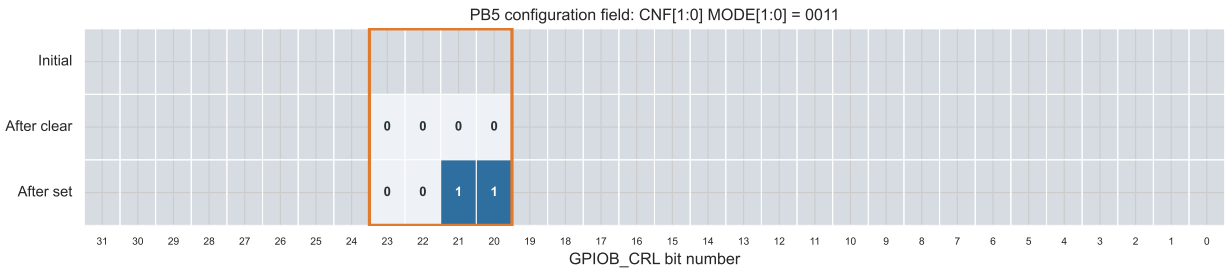

Figure 1: Bit-field progression for GPIOB pin 5. Grey cells are preserved bits.

1.2 Minimum-step code

The clear and set transformations can be combined into one read-modify-write statement:

```
GPIOB->CRL = (GPIOB->CRL & ~(0xFU << 20U)) | (0x3U << 20U);
```

The mask preserves all fields other than PB5, while the encoded value writes `0011` into bits 23:20.

2 Task 2: Evaluating a GPIO Input Bit

The expression is

```
bool Pinx = (GPIOx->IDR & (1U << GPIOx_PIN_N)) != 0U;
```

It evaluates in three operations:

1. `1U << GPIOx_PIN_N` creates a mask with a single one at the selected pin position.
2. Bitwise AND removes every other input bit and retains only the selected pin state.
3. Comparison with zero converts the masked value to `false` or `true`.

For pin 13, the mask is $1 \ll 13 = \texttt{0x2000}$. If `GPIOC->IDR` is `0x2400`, then

$$\texttt{0x2400} \,\&\, \texttt{0x2000} = \texttt{0x2000},$$

which is non-zero, so `Pinx` is `true`. If the register value is `0x0400`, the AND result is zero and `Pinx` is `false`. The `bool` type is supplied by `<stdbool.h>`.

3 Task 3: Button-Controlled LED with IDR and ODR

B1 is read from bit 13 of `GPIOC->IDR`. A press produces the active input level used by the falling-edge board configuration [3]. PA5 is then set or cleared in `GPIOA->ODR`; the other output bits remain unchanged.

Listing 1: Task 3 register-level implementation.

```c
#include "task3_polling.h"

#include <stdbool.h>

#include "main.h"

void Task3_UpdateLedFromButton(void)
{
  bool button_pressed = (GPIOC->IDR & B1_Pin) == 0U;

  if (button_pressed)
  {
    GPIOA->ODR |= LD2_Pin;
  }
  else
  {
    GPIOA->ODR &= ~LD2_Pin;
  }
}
```

The Boolean expression normalizes the button state, after which the two ODR operations directly implement the required LED-on and LED-off behavior.

4 Task 4: Interrupt-Driven 1.5 s Blinking

4.1 Timer calculation

TIM2 receives a 32 MHz clock because APB1 uses a divider of one [2]. A 1 kHz counter tick is formed with a division factor of 32,000:

$$f_{\text{counter}} = \frac{32\,000\,000}{31999 + 1} = 1000\,\text{Hz}.$$

Counting 1,500 ticks produces the requested interval:

$$T_{\text{update}} = \frac{1499 + 1}{1000} = 1.5\,\text{s}.$$

TIM2 values

Timer	`TIM2`
Prescaler register	`31999`
Counter period register	`1499`
Counter mode	Up
Update interrupt	Enabled

The generated initialization applies these values:

```
htim2.Instance = TIM2;
htim2.Init.Prescaler = 31999;
htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
htim2.Init.Period = 1499;
htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
HAL_TIM_Base_Init(&htim2);
```

4.2 External and internal interrupt behavior

The PC13 falling edge calls `HAL_GPIO_EXTI_Callback`. The first accepted press turns LD2 on, resets TIM2, and starts update interrupts. Each TIM2 period callback toggles PA5. The next accepted press stops TIM2 with `HAL_TIM_Base_Stop_IT` and clears PA5. A 30 ms time gate suppresses the extra edges produced by the mechanical button.

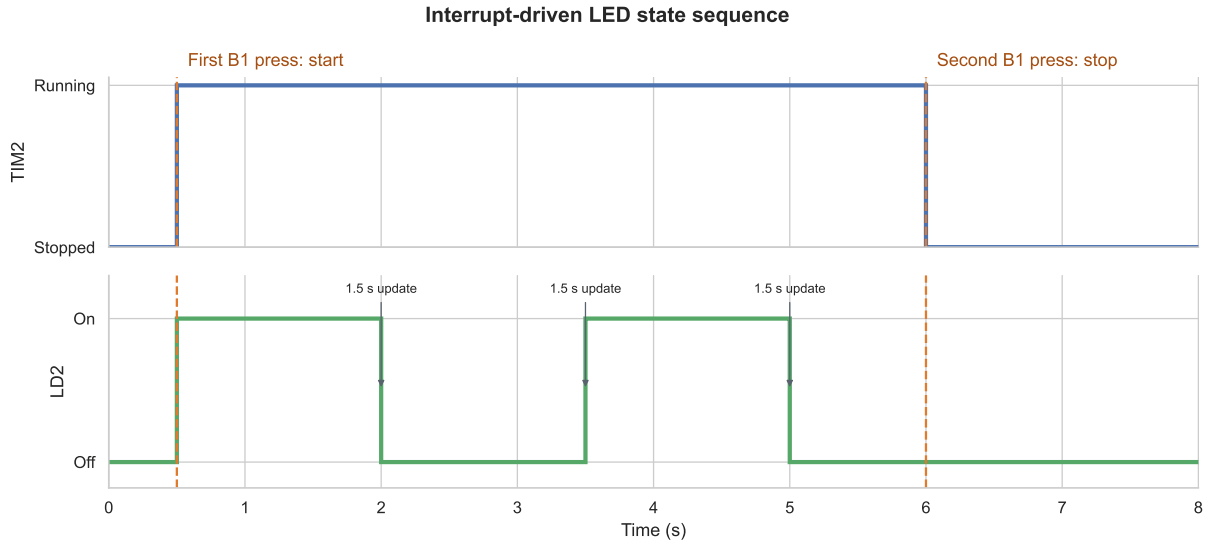

Figure 2: Example Task 4 state sequence. Timer updates toggle LD2 every 1.5 s.

Listing 2: Task 4 application module.

```
#include "lab1_app.h"

#include <stdbool.h>

#include "main.h"

#define BUTTON_DEBOUNCE_MS 30U

static TIM_HandleTypeDef *timer;
static bool blinking;
static uint32_t last_button_tick;
```

```
12
13 static void StartBlinking(void)
14 {
15   blinking = true;
16   GPIOA->ODR |= LD2_Pin;
17   __HAL_TIM_SET_COUNTER(timer, 0U);
18   HAL_TIM_Base_Start_IT(timer);
19 }
20
21 static void StopBlinking(void)
22 {
23   blinking = false;
24   HAL_TIM_Base_Stop_IT(timer);
25   GPIOA->ODR &= ~LD2_Pin;
26 }
27
28 void Lab1_AppInit(TIM_HandleTypeDef *blink_timer)
29 {
30   timer = blink_timer;
31   blinking = false;
32   last_button_tick = HAL_GetTick() - BUTTON_DEBOUNCE_MS;
33   GPIOA->ODR &= ~LD2_Pin;
34 }
35
36 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
37 {
38   uint32_t now;
39
40   if (GPIO_Pin != B1_Pin)
41   {
42     return;
43   }
44
45   now = HAL_GetTick();
46   if ((now - last_button_tick) < BUTTON_DEBOUNCE_MS)
47   {
48     return;
49   }
50   last_button_tick = now;
51
52   if (blinking)
53   {
54     StopBlinking();
55   }
56   else
57   {
58     StartBlinking();
59   }
60 }
61
62 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
63 {
64   if ((htim->Instance == TIM2) && blinking)
65   {
66     GPIOA->ODR ^= LD2_Pin;
67   }
68 }
```

The main loop executes `__WFI()` and contains no delay call. All LED timing is generated by the timer update interrupt, while the button transition is captured by EXTI.

5 Lab Summary

Task 1 used masking, clearing, and shifting to place the required configuration in one GPIO pin field without changing neighbouring fields. Task 2 showed how an IDR mask isolates one sampled input and converts it to a Boolean value. Task 3 applied the same register operations to map the button state directly to the LED state. Task 4 combined a button external interrupt with TIM2 update interrupts: the first press starts a 1.5 s blink sequence, each timer update toggles the LED, and the second press stops the timer and leaves the LED off. Together, the tasks demonstrate the progression from individual bit operations to a small event-driven embedded application.

References

[1] RTCSCA, *Lab 1: Registers, Timers and Interrupts*, 2026–2027.

[2] STMicroelectronics, *STM32L0x3 Reference Manual*, RM0367.
https://www.st.com/resource/en/reference_manual/dm00095744.pdf

[3] STMicroelectronics, *STM32 Nucleo-64 Boards User Manual*, UM1724.
https://www.st.com/resource/en/user_manual/dm00105823.pdf

[4] STMicroelectronics, *STM32F1 Reference Manual*, RM0008.
https://www.st.com/resource/en/reference_manual/cd00171190.pdf